

Chapter 5

Condition variables and semaphores

A mutex answers “who may touch the data now?”. Threads also need to *wait for something*: for an element in a queue, for a thread to finish, for the barber to wake up. Waiting correctly — without burning CPU and without missing the moment the condition becomes true — is what condition variables and semaphores are for. Your `pthread_join` in SWEB is one such wait; P1’s elective asks for mutexes, condition variables and semaphores in SWEB’s user space.

5.1 Waiting: spin or sleep

```
while (queue_empty(q)) {}           /* spinning: burns the CPU, and on one CPU
                                     the producer cannot even run */
```

Waiting should *block*: take the thread off the CPU until another thread says “something changed”. Two problems must be solved together: the waiter must not sleep through the signal (the *lost wake-up*), and the condition must be checked and the data accessed under a mutex.

5.2 Condition variables

```
pthread_mutex_lock(&m);
while (!condition)                /* ALWAYS a loop */
    pthread_cond_wait(&cv, &m);   /* atomically: unlock m and sleep;
                                   on wake-up: lock m again */
/* condition holds, and we hold m */
pthread_mutex_unlock(&m);

/* the other side */
pthread_mutex_lock(&m);
condition = true;
pthread_cond_signal(&cv);         /* wake one waiter (broadcast: all) */
pthread_mutex_unlock(&m);
```

A condition variable has no memory: a signal with nobody waiting is lost — by design. The *condition* lives in ordinary shared variables, protected by the mutex; the condition variable only provides “sleep until somebody might have changed it”.

Why wait must unlock atomically. Suppose the waiter unlocked, and only then went to sleep. In between, the signaller could lock, set the condition, signal (nobody waits yet: lost) and unlock — and the waiter sleeps forever. `pthread_cond_wait` makes “unlock and sleep” one step with respect to other threads. module 05’s `lost_wakeup.c` shows the broken and the fixed version.

Why while, not if. Condition variables in pthreads (and practically everywhere) have *Mesa semantics*: a signal only makes the waiter runnable; by the time it has re-acquired the mutex, another thread may have consumed what it was woken for (a *stolen* wake-up). POSIX also allows *spurious* wake-ups without any signal. Rechecking in a loop handles both. (Under the historical *Hoare semantics* the signaller hands the mutex directly to the woken thread, so the condition is guaranteed to hold — simpler to reason about, more expensive to implement.)

Signal or broadcast? `signal` wakes one waiter: correct when every waiter waits for the same thing and any one of them can proceed. When waiters wait for different conditions on the same variable, or when a change may allow several to proceed, use `broadcast` — a lost signal is a hang, an extra wake-up only costs time.

5.3 The bounded buffer

The pattern to memorise (module 05, `bounded_buffer.c`):

```
void put(struct buf *b, int item) {
    pthread_mutex_lock(&b->m);
    while (b->count == CAP)
        pthread_cond_wait(&b->not_full, &b->m);
    b->item[(b->head + b->count++) % CAP] = item;
    pthread_cond_signal(&b->not_empty);
    pthread_mutex_unlock(&b->m);
}

int get(struct buf *b) {
    pthread_mutex_lock(&b->m);
    while (b->count == 0)
        pthread_cond_wait(&b->not_empty, &b->m);
    int item = b->item[b->head];
    b->head = (b->head + 1) % CAP;
    b->count--;
    pthread_cond_signal(&b->not_full);
    pthread_mutex_unlock(&b->m);
    return item;
}
```

5.4 Semaphores

Dijkstra’s semaphore is a counter with two atomic operations: P (`wait`, `sem_wait`): wait until the value is positive, then decrement; V (`signal`, `sem_post`): increment, waking a waiter if any. Unlike a condition variable, a semaphore *remembers*: a `post` with no waiter is kept in the value. Three roles:

Role	Initial	Use
mutual exclusion	1	<code>wait</code> ; critical section; <code>post</code> (a “binary semaphore” — but anyone may <code>post</code> , unlike a mutex)
signalling	0	thread A <code>waits</code> until thread B <code>posts</code> : “B’s event happened”; the order of the two calls does not matter
counting	N	N identical resources (buffer slots, connections)

The bounded buffer with semaphores: `empty = CAP`, `full = 0`, `mutex = 1`; the producer does `wait(empty)`; `wait(mutex)`; `put`; `post(mutex)`; `post(full)`, the consumer the mirror image. The order of the two `wait`s matters: `wait(mutex)` first and then blocking on `empty` while holding it deadlocks the consumer.

A semaphore is easily built from a mutex and a condition variable (module 05, part A) — and conversely; they are equally powerful.

5.5 Classic problems

Readers–writers. Many readers may read at once, a writer needs exclusive access. The simple solution (readers count themselves; the first reader locks writers out, the last one lets them in) lets a steady stream of readers *starve* writers. A writer-preferring lock makes new readers wait as soon as a writer is waiting (module 05, part B); a fair lock queues everybody in arrival order.

Sleeping barber. A barber sleeps when there are no customers; a customer wakes him, or waits in one of N chairs, or leaves when all chairs are taken. It combines signalling (customer wakes barber, barber calls customer) with a counted resource (chairs) and needs care at the edges: a customer arriving just as the barber decides to sleep must not be lost, and shutdown must wake a sleeping barber (module 05, part C).

Monitors. A monitor bundles data, a mutex that every operation holds, and condition variables — the “one object, one lock” discipline as a language feature (Java’s `synchronized` with `wait/notify`). The bounded buffer above is a monitor written by hand.

In SWEB

SWEB’s `Condition` (`common/source/kernel/Condition.cpp`) is a Mesa condition variable bound to a `Mutex`. `wait` puts the thread on the waiters list *while holding the list’s lock*, releases the mutex, and sleeps. The gap between unlocking the list and `Scheduler::sleep()` is closed from the other side: `Scheduler::wake` yields until the thread really is `Sleeping` before it sets it `Running` — no lost wake-up. `Condition::wait` also asserts that the thread holds no other lock while it sleeps. For `pthread_join`: the joiner checks “is the thread finished?” and waits on such a condition, under the lock that the exiting thread takes to set “finished” and signal.

Key points

- Wait by blocking; the condition lives in shared variables under a mutex.
- `cond_wait` unlocks and sleeps atomically — that is what prevents the lost wake-up.
- Always `while (!condition) wait`: Mesa semantics, stolen and spurious wake-ups.
- When unsure, `broadcast`.
- Semaphores remember posts: mutex (1), signal (0), count (N).
- Readers–writers: watch for writer starvation; barber: watch the edges and the shutdown.